

Summary of Tool Use

Problem and Motivation

The rapid advancement of Large Language Models (LLMs) has significantly broadened their capabilities in areas such as mathematical reasoning and program synthesis. Despite these gains, a critical limitation persists: effectively interacting with external tools via API calls. This shortcoming becomes particularly evident when contrasted with state-of-the-art closed-source LLMs like ChatGPT, which can handle tool use more reliably. Two main issues stand out: the frequent generation of inaccurate function arguments and a tendency to hallucinate nonexistent or incorrect API usage, undermining the practical deployment of LLMs [16, 17].

Bridging this gap is vital for multiple reasons. First, stronger tool-use capabilities would expand LLMs' functional range, enabling them to handle more sophisticated tasks. Such an expansion moves the field closer to artificial general intelligence (AGI), where models can adapt to a dynamic environment and self-correct when making tool-based decisions. Additionally, improved reliability in API calls is essential for deploying LLMs in real-world settings, where automated code generation and seamless tool interaction can streamline or even eliminate repetitive tasks in software development pipelines [16, 17].

Several promising frameworks aim to tackle these challenges. ToolLLM proposes a unified approach encompassing data construction, model training, and evaluation to hone the tool-use proficiencies of open-source LLMs. Notably, it builds on a large-scale dataset, ToolBench, and uses a depth-first search-based decision tree algorithm to improve both the reasoning depth and retrieval of relevant APIs [17]. Gorilla, on the other hand, centers on fine-tuning LLaMA-based models to surpass even GPT-4 in generating valid API calls. It leverages a document retrieval system at inference time to adapt to updated API documentation, thus reducing the likelihood of hallucinations [16]. These advancements collectively mark a significant step toward robust, open-source LLMs that can confidently handle real-world tool-use scenarios, bridging the gap with leading closed-source systems.

Related Works

The integration of LLMs with external tools has evolved through multiple research directions, spanning tool learning paradigms, instruction tuning, and evaluation frameworks. Early efforts focused on enabling LLMs to interact with specific tools through *self-supervised learning*. For instance, Toolformer [1] demonstrated that LLMs could autonomously learn to invoke calculators, translators, and search engines by augmenting their training data with API call demonstrations. While groundbreaking, Toolformer's scope was limited to a small set of predefined tools and lacked systematic evaluation metrics. Subsequent works like API-Bank [2] expanded this paradigm by introducing task-specific datasets for API invocation, though their coverage remained narrow (e.g., 53 APIs) and focused on single-tool scenarios.

The rise of *instruction tuning* further bridged the gap between general-purpose LLMs and tool use. Models like Alpaca [3] and Vicuna [4] leveraged self-instructed data to improve instruction-following capabilities, but their tool manipulation skills remained underdeveloped. To address this, DocPrompting [5] proposed retrieving API documentation during inference to guide code generation, reducing hallucination by grounding responses in structured API specifications. Meanwhile, ToolAlpaca [6] simulated 3,938 tool-use cases across 400 APIs, demonstrating that fine-tuning on synthetic data could enhance LLMs' ability to parse parameters and execute API calls. However, these approaches often assumed static API documentation and struggled with real-world complexities like overlapping functionalities or version updates.

A critical challenge in tool learning lies in *multi-step reasoning and planning*. ReACT [7] introduced a framework combining reasoning (Chain-of-Thought) and acting (API calls), enabling iterative interactions with tools while incorporating environmental feedback. This approach improved robustness but suffered from error propagation, as incorrect early steps could derail entire workflows. Reflexion [8] mitigated this by enabling LLMs to reflect on past failures and adjust strategies dynamically. Concurrently, Tree-of-Thought (ToT) [9] explored branching reasoning paths to expand decision spaces, though its computational cost limited scalability. These methods highlighted the need for efficient exploration-retraction mechanisms in complex tool-use scenarios.

The evaluation of tool-equipped LLMs has also seen innovation. Traditional metrics like BLEU or exact match often failed to capture functional correctness, leading to proposals like *Abstract Syntax Tree (AST) matching* [10] to validate API call structures. Other works emphasized *dynamic evaluation*: WebGPT [11] tested models' ability to browse the web and synthesize answers, while Visual ChatGPT [12] assessed multi-

modal tool integration. However, many benchmarks lacked diversity in API types or real-time constraints, such as handling deprecated endpoints or version mismatches. Recent efforts like APIBench [13] introduced domain-specific metrics for machine learning APIs, though broader generalization to RESTful or domain-agnostic tools remained underexplored.

Efforts to scale tool learning to *massive API ecosystems* have introduced retrieval-augmented architectures. For example, ToolQA [14] trained a neural retriever to recommend relevant APIs from thousands of candidates, reducing manual annotation overhead. Similarly, RestGPT [15] focused on RESTful API composition, enabling models to chain calls across services like weather forecasts and e-commerce platforms. These systems highlighted the importance of *retrieval precision* and *context-aware planning* in open-ended environments. Challenges persist in balancing specificity (e.g., handling API parameter constraints) and generality (e.g., adapting to unseen tools), particularly when user instructions involve implicit preferences like cost or latency.

Solution Overview

ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs

This study presents ToolLLM, a framework designed to enhance the tool-use capabilities of open-source Large Language Models (LLMs) by enabling them to interact effectively with real-world APIs. Existing instruction tuning approaches primarily focus on language tasks while neglecting tool utilization, leaving open-source models significantly behind proprietary models like ChatGPT. To address this gap, ToolLLM introduces ToolBench, a large-scale instruction-tuning dataset built through a three-stage process: (i) API collection, which gathers 16,464 RESTful APIs across 49 categories from RapidAPI; (ii) instruction generation, where ChatGPT produces diverse prompts incorporating both single-tool and multi-tool interactions; and (iii) solution path annotation, in which ChatGPT generates valid API call sequences for each instruction. This dataset enables LLMs to learn structured tool-use capabilities, forming the foundation for ToolLLaMA, a fine-tuned version of LLaMA-2 designed for API execution, as illustrated in Figure 1.

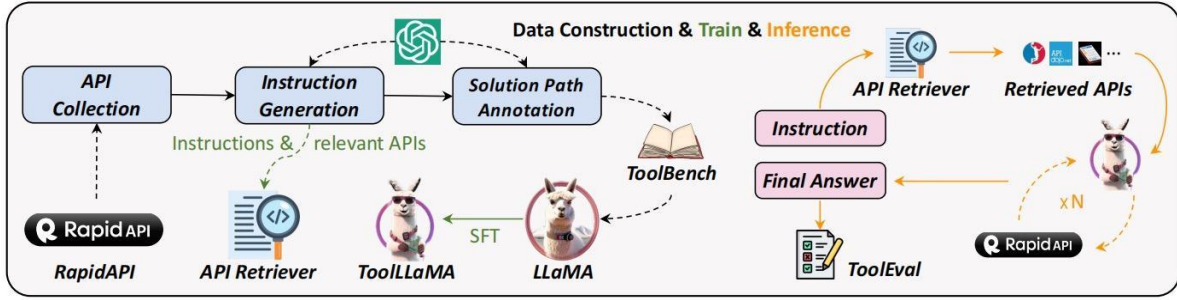


Figure 1: Three phases of constructing ToolBench and how to train our API retriever and ToolLLaMA.

To enhance reasoning efficiency, DFSDT (Depth-First Search-based Decision Tree) is introduced to overcome the limitations of CoT and ReACT, which struggle with error propagation and limited exploration. DFSDT expands the search space by evaluating multiple solution paths, enabling models to iteratively refine API calls rather than following a single fixed trajectory. This improves ToolLLaMA’s ability to recover from errors and explore better execution strategies, leading to more effective tool utilization and decision-making in API selection and sequencing, as shown in Figure 2. Additionally, a neural API retriever is integrated to recommend relevant APIs dynamically, reducing reliance on predefined mappings and improving generalization to unseen APIs.

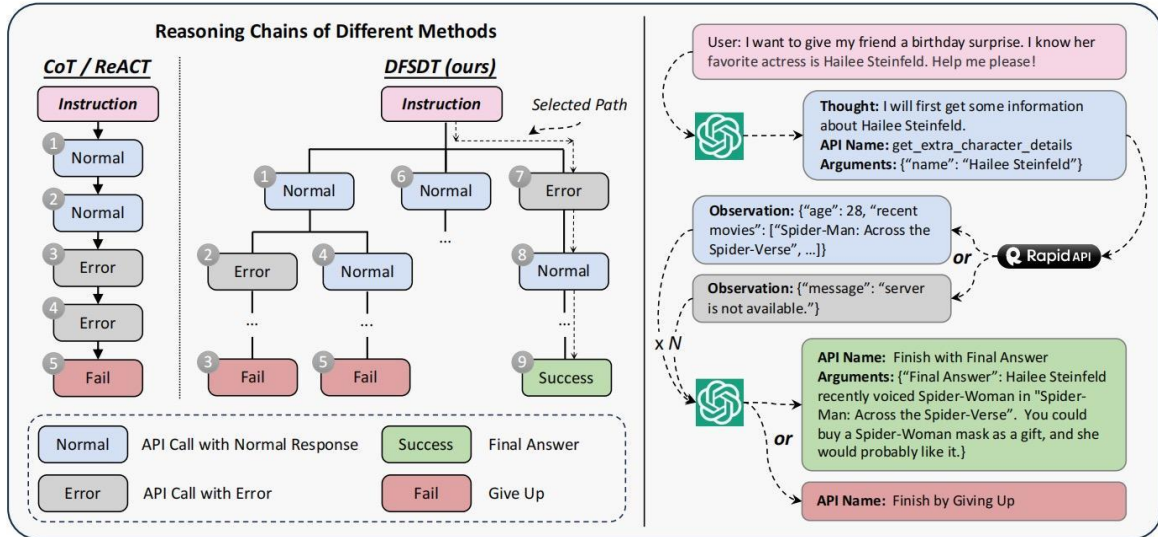


Figure 2: A comparison of DFSDT and conventional CoT or ReACT during model reasoning.

Experiments show that ToolLLaMA with DFSDT outperforms Text-Davinci-003 and Claude-2, achieving performance close to ChatGPT while demonstrating strong generalization to unseen instructions and APIs. Integrating an API retriever further improves pass rate and win rate, highlighting the benefits of automatic API selection. In contrast, models relying solely on prompt engineering, such as Vicuna and Alpaca, fail in API-based tasks, underscoring the limitations of current instruction-tuning approaches. These results confirm ToolLLM’s effectiveness in bridging the gap between open-source and proprietary LLMs, with its dataset, model, and evaluation framework publicly available for future research.

Gorilla: Large Language Model Connected with Massive APIs

Large Language Models (LLMs) have demonstrated remarkable advancements across various domains, including mathematical reasoning and program synthesis. However, their ability to effectively utilize tools via API calls remains an open challenge. Even state-of-the-art models like GPT-4 struggle with API calls due to a lack of awareness regarding available APIs and their proper usage within frequently updated toolsets.

To address this issue, Gorilla, a fine-tuned LLaMA model, is introduced to enhance API-call generation. As shown in Figure 1, Gorilla is trained on APIBench, a comprehensive dataset consisting of APIs from HuggingFace, TorchHub, and TensorHub, covering over 1,600 machine-learning APIs. Gorilla leverages Retriever-Aware Training (RAT), a novel technique that enables the model to dynamically retrieve and utilize API documentation at inference time, ensuring accurate and up-to-date API usage. This approach significantly mitigates hallucinations—one of the most common challenges in LLM-generated API calls.

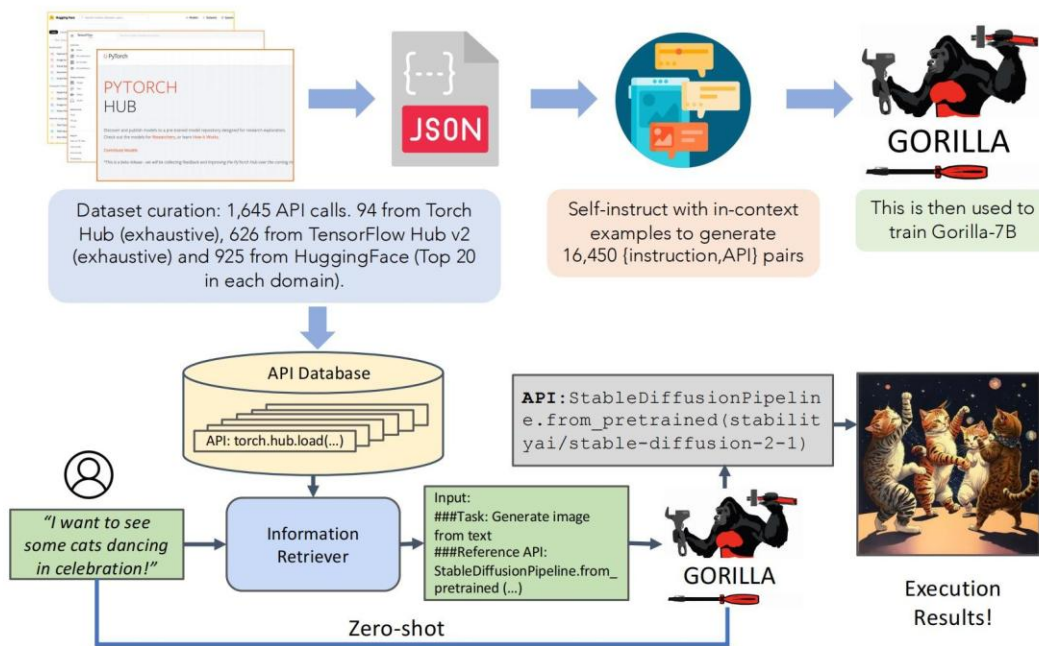


Figure 1: Gorilla: A system for enabling LLMs to interact with APIs. The upper half represents the training procedure. During inference (lower half), Gorilla supports two modes - with retrieval, and zero-shot.

To verify API correctness, Gorilla employs AST Sub-Tree Matching (Figure 2), a method that evaluates API functional equivalence by comparing generated API calls to structured reference datasets. Figure 3 presents empirical results demonstrating that Gorilla outperforms GPT-4, GPT-3.5, and Claude in API-call accuracy, particularly on TorchHub and HuggingFace APIs, while achieving competitive performance on TensorFlow Hub APIs.

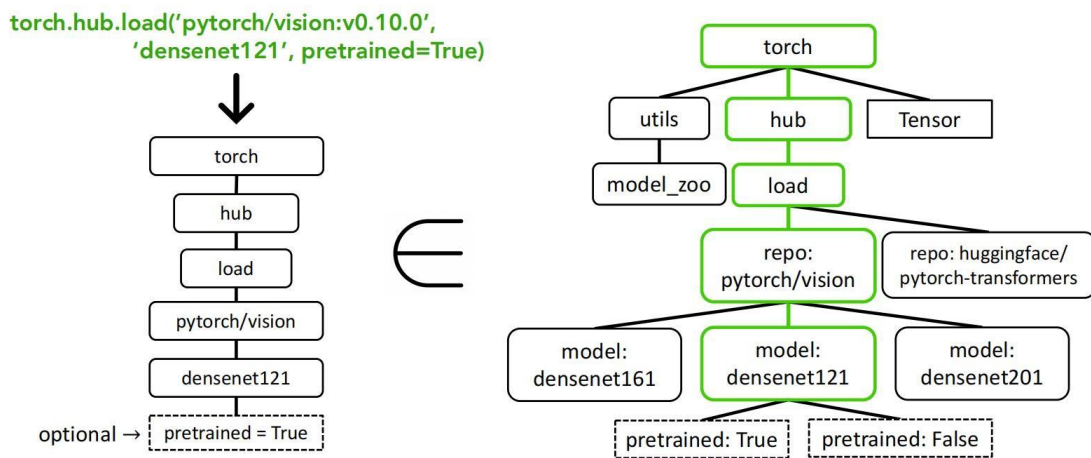


Figure 2: AST Sub-Tree Matching to evaluate API calls.

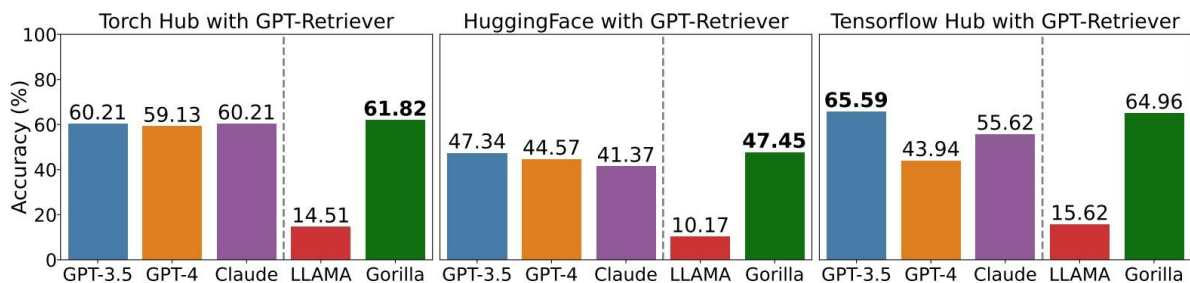


Figure 3: Accuracy with GPT-retriever. Gorilla outperforms on Torch Hub and Hugging-Face while matching performance on Tensorflow Hub for all existing state-of-the-art LLMs.

By integrating retrieval-based learning with instruction tuning, Gorilla offers a scalable and reliable solution for improving LLM-driven API interactions. Its ability to adapt to API changes in real-time enables more accurate tool usage in real-world applications, demonstrating the potential of retrieval-augmented LLMs in software development and automation.

Limitations

Despite the promising achievements of LLMs for tool use, they still have the following limitations:

1. The current scope of tools is quite narrow, primarily confined to the APIs of the digital world. With the rise of embodied AI, the application of tools in the physical world is increasingly important for the development of artificial intelligence. Therefore, the category of tools should not be limited to the digital world alone but should consider some real tools in the physical world.
2. The current annotation/evaluation pipeline is not rigorous and may lead to false impressions about the model's performance. Since the evaluation of tool use currently relies mainly on larger LLMs or human assessments, it can result in illusions caused by LLMs or subjective biases present in human evaluations.
3. Most of the technical ideas in this work are from past works, with the exception of DFSDT, which is a simple application of DFS to prompting. In this sense, the current work's technical contribution is low.
4. The application scenarios of the model mainly focus on the iid setting, while real-world testing scenarios often need to test many tools OOD from the training data. Therefore, the model's generalization capability in OOD scenarios is still limited.

5. In the real world, APIs are continuously updated and iterated. Some old APIs will no longer be used or will undergo functional changes, and the current model has not yet taken into account these dynamic changes in APIs.

Future Research Directions

Combining the introduction of the article and the discussion of limitations, we believe that future research directions mainly include the following aspects:

1. Expansion and integration of tool interaction in the physical world: In the future, we need to break through the limitations of digital APIs and build an embodied intelligent system that integrates physical and digital. By integrating robots, IoT devices and multimodal perception technology, we develop an interactive framework that supports tactile feedback, visual positioning and natural language linkage; establish a structured knowledge base covering the physical properties, operating specifications and safety constraints of real tools, and use the cognitive ability of pre-trained models in high-fidelity simulation environments (such as NVIDIA Isaac Sim) to design a Sim2Real migration strategy based on reinforcement learning to achieve a smooth transition from virtual training to real operations.
2. Construction of a scientific evaluation system: It is necessary to establish a more rigorous evaluation methodology, develop a tool call benchmark that supports automated verification (such as ToolBench-Engine), and quantitatively evaluate indicators such as API call correctness, physical operation target achievement, and energy efficiency; introduce adversarial sample generation technology to construct a test set of edge cases that include functional ambiguity and parameter conflicts, and combine neural symbolic methods to detect cognitive biases in the model decision chain, and use formal verification methods (such as logical constraint verification) to improve the objectivity and interpretability of evaluation results.
3. Continuous adaptability in a dynamic environment: In response to the challenge of dynamic API updates in real-world scenarios, it is necessary to design an incremental lifelong learning framework to achieve collaborative storage of new and old tool knowledge through elastic weight consolidation (EWC) and memory playback technology; develop an API version-sensitive mechanism and use metadata vector similarity calculation to achieve version adaptive selection; build a collaborative evolution platform for tool developers and LLM, establish an automated link between real-time push of API changes and model fine-tuning, and ensure the model's ability to continuously track the evolution of the tool ecosystem.

4. Breakthrough in OOD generalization capability: To improve the generalization performance of the model when facing new tools, a "meta-skill library" can be constructed through functional abstraction, guiding the model to combine meta-skills to solve unknown tool tasks; introducing causal reasoning technology modeling tools to use causal graphs to distinguish between core functional attributes and surface features; designing cross-domain analogy transfer algorithms to migrate known tool call logic to new functionally equivalent tools.

Summary of Class Discussion

In the class, there was an intense discussion about the use of "tool using," focusing primarily on the current limitations of LLMs for tool using. First, there was a conversation centered around the dataset of the tool, highlighting that the current API dataset has limited applicability and could be expanded to include pipeline functions, such as Python programming and formal languages.

Furthermore, the methods of evaluating tool usage were discussed, with current evaluations typically conducted by GPT models or humans. The group noted that GPT evaluations tend to have significant biases and lack efficiency, whereas human evaluations could improve in quality by considering more semantic information of the tools.

Additionally, there has been some discussion about the clarification of detailed experimental results. Referring to the original implementation and codes can offer more insights about these.

References

- [1] Schick, Timo, et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems*, 2023.
- [2] Li, Minghao, et al. "API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs." *arXiv preprint* arXiv:2304.08244, 2023.
- [3] Taori, Rohan, et al. "Stanford Alpaca: An Instruction-Following LLaMA Model." 2023.
- [4] Chiang, Wei-Lin, et al. "Vicuna: An Open-Source Chatbot Impressing GPT-4." 2023.
- [5] Zhang, Han, et al. "DocPrompting: Generating Code by Retrieving the Docs." 2023.
- [6] Chen, Zhenfei, et al. "ToolAlpaca: Generalized Tool Learning for Language Models." 2023.
- [7] Yao, Shunyu, et al. "ReACT: Synergizing Reasoning and Acting in Language Models." 2022.
- [8] Shinn, Noah, et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." 2023.
- [9] Yao, Shunyu, et al. "Tree of Thoughts: Deliberate Problem Solving with LLMs." 2023.

- [10] Nijkamp, Erik, et al. "CodeGen: An Open Large Language Model for Code." 2022.
- [11] Nakano, Reiichiro, et al. "WebGPT: Browser-Assisted Question-Answering with Human Feedback." 2021.
- [12] Wu, Chenfei, et al. "Visual ChatGPT: Talking, Drawing, and Editing with Visual Tools." 2023.
- [13] Zhao, Yao, et al. "APIBench: A Dataset for Tool-Augmented LLMs in Machine Learning." 2023.
- [14] Chen, Zhenfei, et al. "ToolQA: A Dataset for LLM Question Answering with External Tools." 2023.
- [15] Qin, Yujia, et al. "RestGPT: Connecting LLMs with RESTful APIs." 2023.
- [16] Patil, Shishir G., et al. "Gorilla: Large language model connected with massive apis." Advances in Neural Information Processing Systems 37 (2025): 126544-126565.
- [17] Qin, Yujia, et al. "Toollm: Facilitating large language models to master 16000+ real-world apis." arXiv preprint arXiv:2307.16789 (2023).